PROJECT REPORT

ComicCraft

AI Comic Story Creator using Gemini Models

A Generative AI web application for creating structured comic stories, characters, scenes, dialogue and visual prompts.

Project Type	Generative AI / Web Application
AI Platform	Google Gemini API
Backend	Python + Flask
Frontend	HTML5 + CSS3 + JavaScript
Output	Comic script + panel-by-panel visual prompts

Prepared as a complete academic project report and implementation blueprint.

2026

CERTIFICATE

This is to certify that the project entitled **“ComicCraft – AI Comic Story Creator using Gemini Models”** is presented as an academic software project demonstrating the application of Generative AI for automated comic-story creation.

Student Name	_____
Register Number	_____
Department	B.Sc. Computer Science
Institution	_____
Academic Year	2026–2027

Project Guide Signature: _____

Head of Department: _____

ABSTRACT

ComicCraft is a Generative AI based application designed to help users create original comic stories from a simple idea. A user enters a concept such as genre, theme, characters, setting and desired number of panels. The application sends a structured prompt to a Gemini model and receives a coherent story plan containing a title, character descriptions, panel narration, dialogue, captions and visual prompts. The result is presented in a comic-friendly layout and can be extended with AI-generated artwork.

The project demonstrates prompt engineering, structured output, web application development, API integration, validation, error handling and responsible AI usage. Google's current Gemini API documentation supports content generation and image generation, making it suitable for this multimodal workflow.

TABLE OF CONTENTS

1	Introduction
2	Problem Statement
3	Objectives
4	Existing and Proposed System
5	Requirements
6	Architecture
7	Modules
8	Data Model
9	Prompt Engineering
10	Implementation
11	UI Design
12	Image Generation
13	Testing
14	Security & Responsible AI
15	Advantages & Limitations
16	Future Scope
17	Conclusion & References
18	Appendix – Source Code

1. INTRODUCTION

Comics combine storytelling, visual composition, dialogue and sequential art. Creating a comic traditionally requires idea development, character design, script writing, panel planning and illustration. ComicCraft explores how Generative AI can reduce the initial creative workload by transforming a short user idea into a structured comic plan.

The application uses a web interface to collect creative requirements. The backend validates the input, constructs a controlled prompt, calls the Gemini API, parses the response and renders the resulting comic script. The architecture keeps text generation and image generation modular.

2. PROBLEM STATEMENT

Aspiring comic creators may have ideas but struggle to convert them into a complete sequential story. Manual story planning takes time, and maintaining character consistency across multiple panels can be difficult. ComicCraft addresses this by providing an AI-assisted pipeline that converts a high-level concept into an organized comic script and visual plan.

3. OBJECTIVES

• Build a simple web application for AI-assisted comic creation.

• Integrate Gemini securely through a server-side backend.

• Use structured prompting to obtain predictable story components.

• Generate panel-by-panel dialogue, narration and visual prompts.

• Handle invalid input, API failures and malformed AI responses.

• Provide a design that can later support generated comic images and PDF export.

• Demonstrate practical Generative AI application development.

4. EXISTING AND PROPOSED SYSTEM

Existing Approach

A conventional workflow requires separate tools for brainstorming, script writing, character planning and artwork. Users manually transfer information between these tools, increasing effort and the possibility of inconsistencies.

Proposed Approach

ComicCraft combines the initial stages into one application. The user supplies creative constraints once, and the system creates a structured comic blueprint. Gemini handles natural-language generation while the application controls schema, validation and presentation.

5. SYSTEM REQUIREMENTS

Hardware

• Dual-core processor or better.

• 4 GB RAM minimum; 8 GB recommended.

• At least 500 MB application storage.

• Internet connection for Gemini API requests.

Software

Component	Recommended
OS	Windows / Linux / macOS
Language	Python 3.10+
Backend	Flask
Frontend	HTML5, CSS3, JavaScript
AI SDK	google-genai
Environment	Virtual environment + .env
Optional DB	SQLite
Browser	Chrome / Edge / Firefox

Google's current Gemini getting-started documentation describes obtaining an API key through Google AI Studio and installing the Python SDK with **pip install -U google-genai**.

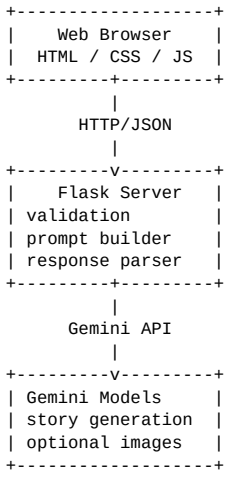
6. SYSTEM ARCHITECTURE

ComicCraft follows a three-layer architecture: presentation, application logic and AI/data services.

Layer	Responsibility
Presentation	Form input, comic preview, loading/error states and export controls.
Application	Validation, prompt construction, Gemini calls, response parsing.
AI / Data	Gemini story/image generation and optional SQLite project storage.

Data Flow

Idea → Frontend validation → Flask → Prompt builder → Gemini → JSON validation → Comic renderer → Optional image generation → Export.



7. FUNCTIONAL MODULES

Module	Function
User Input	Collect idea, genre, tone, setting, characters and panel count.
Prompt Builder	Converts requirements into a strict structured prompt.
Story Generator	Calls Gemini and creates title, premise and panel data.
Consistency Controller	Repeats character descriptions in visual prompts.
Comic Renderer	Displays panels with narration, dialogue and image placeholders.
Image Generator	Optionally creates artwork from visual prompts.
Export	Saves or prints the generated comic.
Error Handler	Handles empty input, API failure and invalid AI output.

8. DATA MODEL

```
Project
id, title, idea, genre, tone, panel_count, story_json, created_at

Character
id, project_id, name, description, visual_traits

Panel
id, project_id, panel_number, narration, dialogue,
visual_prompt, image_path
```

SQLite is optional in the first version. It can be added to store generated projects, characters and panel metadata so users can reopen previous comics.

9. PROMPT ENGINEERING

The model should not receive only the user's sentence. ComicCraft defines a role, inputs, constraints and an explicit output schema. Structured output makes the generated story easier for the backend to parse and render.

```
SYSTEM:
You are ComicCraft, a professional comic story planner.
Create original, coherent, general-audience comic stories.

USER:
Idea: {idea}
Genre: {genre}
Tone: {tone}
Setting: {setting}
Characters: {characters}
Panels: {panel_count}

RULES:
1. Return valid JSON only.
2. Keep character names and traits consistent.
3. Every panel must advance the story.
4. Include narration, dialogue and visual_prompt.
5. Do not return markdown fences.

OUTPUT:
{
  "title": "...",
  "premise": "...",
  "characters": [...],
  "panels": [
    {
      "panel": 1,
      "narration": "...",
      "dialogue": [...],
      "visual_prompt": "..."
    }
  ]
}
```

Why Structured Prompts Matter

• They reduce ambiguity in the expected output.

• They make JSON parsing more reliable.

• They improve consistency across panels.

• They make the application easier to test.

• They allow the frontend to render predictable fields.

10. IMPLEMENTATION

Project Structure

```
comiccraft/
■■■ app.py
■■■ requirements.txt
■■■ .env
■■■ templates/
■   ■■■ index.html
■■■ static/
■   ■■■ style.css
■   ■■■ app.js
■■■ generated/
■■■ README.md
```

Installation

```
python -m venv venv

# Windows
venv\Scripts\activate

# Linux/macOS
source venv/bin/activate

pip install flask python-dotenv google-genai
```

Environment

```
# .env
GEMINI_API_KEY=YOUR_API_KEY_HERE
```

The API key must stay on the server and must never be committed to source control or exposed in browser JavaScript.

Core Flask Backend

```
import os, json
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from google import genai

load_dotenv()
app = Flask(__name__)
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

def build_prompt(d):
    return (
        "You are ComicCraft, a professional comic story planner. "
        "Return valid JSON only.\n"
        f"Idea: {d['idea']}\n"
        f"Genre: {d.get('genre','Adventure')}\n"
        f"Tone: {d.get('tone','Fun')}\n"
        f"Setting: {d.get('setting','Unspecified')}\n"
        f"Characters: {d.get('characters','Create suitable characters')}\n"
        f"Panels: {d.get('panel_count',6)}\n"
        "Create title, premise, characters and panels. "
        "Each panel needs narration, dialogue and visual_prompt."
    )

@app.get("/")
def home():
    return render_template("index.html")

@app.post("/generate")
def generate():
    data = request.get_json(silent=True) or {}
    if not str(data.get("idea","")).strip():
        return jsonify({"error":"Story idea is required."}), 400

    response = client.models.generate_content(
        model="gemini-3.8-flash",
        contents=build_prompt(data)
    )

    try:
        result = json.loads(response.text)
    except json.JSONDecodeError:
        return jsonify({"error":"AI returned invalid story format."}), 502

    return jsonify(result)
```

```
if __name__ == "__main__":
    app.run(debug=False)
```

The code pattern follows Google's current Python Gemini SDK examples for content generation. Model identifiers and API capabilities should be checked against the current documentation before deployment.

11. USER INTERFACE DESIGN

Main Screen

The home page should contain the ComicCraft title, story idea text area, genre selector, tone selector, setting field, character field, panel-count selector and Generate Comic button.

```
<form id="comicForm">
  <textarea id="idea" placeholder="Enter your comic idea"></textarea>
  <select id="genre">
    <option>Adventure</option>
    <option>Comedy</option>
    <option>Fantasy</option>
    <option>Science Fiction</option>
    <option>Mystery</option>
  </select>
  <select id="tone">
    <option>Fun</option>
    <option>Dark</option>
    <option>Emotional</option>
    <option>Inspirational</option>
  </select>
  <input id="setting" placeholder="Setting">
  <input id="characters" placeholder="Characters">
  <input id="panel_count" type="number" min="3" max="12" value="6">
  <button type="submit">Generate Comic</button>
</form>
<div id="result"></div>
```

Frontend Flow

• Collect user data.

• Send JSON to /generate.

• Show a loading state.

• Receive structured response.

• Render title, characters and panels.

• Allow regeneration or export.

12. OPTIONAL IMAGE GENERATION

A richer ComicCraft version can turn each panel's visual prompt into artwork. Google's current Gemini image-generation documentation describes Gemini image models for text-to-image and image-editing workflows. It currently identifies Gemini 3.1 Flash Image as a general-purpose image-generation option, with other models for different performance and quality needs.

```
from google import genai
import base64

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.1-flash-image",
    input="Create a comic panel: " + panel["visual_prompt"]
)

if interaction.output_image:
    with open("generated/panel_01.png", "wb") as f:
        f.write(base64.b64decode(
            interaction.output_image.data
        ))
```

Recommended workflow: generate the text story first, preserve the same character descriptions, then generate panel artwork. This reduces continuity problems and makes the visual prompts easier to review.

Important Note

Image generation capabilities, model names, quotas and pricing can change. Verify the current Gemini documentation before implementation or deployment. Google also advises that users have appropriate rights to images they upload for editing.

13. TESTING

ID	Scenario	Expected Result
TC01	Empty idea	Validation message; no API request.
TC02	Valid 6-panel request	Structured six-panel story.
TC03	Invalid panel count	Rejected or limited to allowed range.
TC04	Missing API key	Clear configuration error.
TC05	API failure	Retry/error message without crash.
TC06	Malformed AI output	JSON error handled safely.
TC07	Special characters	No script/HTML injection.
TC08	Very long input	Input length limit applied.
TC09	Repeated character names	Consistent character identity.
TC10	Image generation off	Text-only workflow still works.

Performance Metrics

Record average generation time, API request count, response size, failure rate and image-generation latency across 10–20 test runs.

14. SECURITY AND RESPONSIBLE AI

• Keep the Gemini API key in environment variables.

• Apply input length limits and rate limiting.

• Escape generated text before inserting it into HTML.

• Never expose API secrets or raw server stack traces.

• Validate AI output before rendering or saving it.

• Use original content or content for which the user has appropriate rights.

• Add audience-appropriate safety and moderation controls.

• Treat generated content as AI-assisted output that may require human editing.

Google's Gemini image-generation documentation specifically notes the need to have appropriate rights to uploaded images and to follow applicable usage requirements.

15. ADVANTAGES AND LIMITATIONS

Advantages

• Fast idea-to-script workflow.

• Structured sequential storytelling.

• Useful for students and hobby creators.

• Expandable to multimodal comic creation.

• Demonstrates practical API integration.

Limitations

• AI may produce continuity or logic errors.

• Character appearance can vary between images.

• API limits and pricing may change.

• Image generation adds latency and usage.

• Human editing remains useful for final-quality comics.

16. FUTURE SCOPE

• User accounts and saved comic projects.

• SQLite/PostgreSQL project history.

• Character reference sheets for visual consistency.

• Multi-page comic PDF export.

• Automatic speech-bubble placement.

• Voice narration of dialogue.

• Tamil + English multilingual comic generation.

• One-panel regeneration instead of regenerating the whole comic.

• Collaborative editing and sharing.

• Additional style presets where supported by the selected image model.

17. CONCLUSION

ComicCraft demonstrates how Generative AI can be integrated into a practical web application rather than being used only as a standalone chatbot. The system separates input, prompt construction, model generation, validation and presentation into understandable modules, making it suitable as a student project and providing a clear path toward a richer multimodal comic platform.

Core workflow: idea → structured story → characters → panels → visual prompts → optional artwork → comic output.

REFERENCES

1. Google AI for Developers – Gemini API: Generating Content.

2. Google AI for Developers – Gemini API: Getting Started.

3. Google AI for Developers – Gemini API: Image Generation.

4. Google AI for Developers – Gemini image model selection and guidance.

18. APPENDIX – CORE SOURCE CODE

requirements.txt

```
Flask
python-dotenv
google-genai
```

index.html – minimal version

```
<!doctype html>
<html>
<head><title>ComicCraft</title></head>
<body>
<h1>ComicCraft</h1>
<form id="comicForm">
<textarea id="idea" required
 placeholder="Describe your comic idea"></textarea>
<input id="genre" value="Adventure">
<input id="tone" value="Fun">
<input id="setting" placeholder="Setting">
<input id="characters" placeholder="Characters">
<input id="panel_count" type="number" min="3" max="12" value="6">
<button>Generate Comic</button>
</form>
<pre id="result"></pre>
<script>
comicForm.onsubmit = async (e) => {
  e.preventDefault();
  const payload = {
    idea: idea.value, genre: genre.value, tone: tone.value,
    setting: setting.value, characters: characters.value,
    panel_count: Number(panel_count.value)
  };
  const r = await fetch("/generate", {
    method:"POST",
    headers:{"Content-Type":"application/json"},
    body:JSON.stringify(payload)
  });
  const data = await r.json();
  result.textContent = r.ok
    ? JSON.stringify(data, null, 2)
    : data.error;
};
</script>
</body>
</html>
```

How to Run

```
1. Create the folder structure in this report.
2. Create and activate a Python virtual environment.
3. Install requirements.txt.
4. Add GEMINI_API_KEY to .env.
5. Run: python app.py
6. Open: http://127.0.0.1:5000
7. Enter a comic idea and click Generate Comic.
```

Implementation note: model names and API capabilities evolve. Before submission, verify the currently available model identifier and API behavior in Google's official Gemini documentation.